

Reto final

Bryan Michael Villa Camacho
Carrera de Computación
Universidad Técnica Particular de Loja
Loja, Ecuador
bmvilla@utpl.edu.ec

Resumen—La preparación manual de recursos didácticos universitarios —resúmenes, glosarios, preguntas de comprensión y actividades de reflexión— representa una inversión de tiempo significativa que se repite cada período académico. Este trabajo presenta el diseño de una herramienta automatizada que, a partir del plan docente en formato PDF, genera dichos recursos mediante un pipeline de cuatro prompts encadenados sobre la API de Gemini. El sistema extrae y segmenta el texto con `pypdf`, construye una base de conocimiento estructurada en Python, invoca al modelo `gemini-1.5-flash` con salida estructurada en JSON y exporta los resultados a un documento Word con `python-docx`. La calidad se evalúa mediante el paradigma LLM-as-a-Judge complementado con validación del docente titular.

Index Terms—pipeline de prompts, Gemini API, generación de recursos didácticos, LLM-as-a-Judge, prompt chaining, python-docx

I. INTRODUCCIÓN

Los modelos de lenguaje de gran escala (LLM) han demostrado una capacidad creciente para generar contenido textual estructurado a partir de instrucciones precisas [5]. Sin embargo, su uso directo sin un diseño instruccional adecuado produce resultados inconsistentes en calidad y formato, especialmente en tareas educativas que requieren organización taxonómica y precisión conceptual [3].

Este proyecto aborda el problema de la generación automática de recursos didácticos universitarios mediante un pipeline de prompts encadenados sobre la API de Gemini. El sistema toma como entrada el plan docente de una asignatura en formato PDF y produce, por cada unidad temática, cuatro recursos: resumen ejecutivo, glosario de términos, preguntas de comprensión con niveles de la taxonomía de Bloom y una actividad de reflexión. Los recursos se exportan automáticamente a un documento Word con `python-docx` y su calidad se valida mediante el paradigma LLM-as-a-Judge [2].

II. ESTADO DEL ARTE

II-A. LLM aplicados a la generación de contenido educativo

La aplicación de modelos de lenguaje a la generación de contenido educativo ha mostrado resultados consistentes en tareas de síntesis temática, formulación de preguntas y construcción de glosarios [4]. Investigaciones recientes demuestran que la calidad pedagógica del output depende en mayor medida del diseño instruccional del prompt que de la escala del modelo [3]. La especificación del rol del modelo,

las restricciones de formato y la inclusión de ejemplos determinan la coherencia del contenido generado para audiencias universitarias, lo que fundamenta la decisión de diseño central del presente proyecto: invertir el esfuerzo en la ingeniería de prompts antes que en la complejidad arquitectónica del sistema.

II-B. Prompt chaining y salida estructurada

El encadenamiento de prompts descompone tareas complejas en sub tareas atómicas con instrucciones específicas, reduciendo la ambigüedad de cada llamada al modelo [6]. La técnica de Chain-of-Thought (CoT) [1] complementa este enfoque al forzar al modelo a generar razonamiento intermedio antes de producir la respuesta final, lo que mejora la coherencia en tareas con organización taxonómica como la formulación de preguntas según los niveles de Bloom. La exigencia de salida en JSON en cada prompt elimina la variabilidad de formato entre ejecuciones y permite que el módulo exportador procese los recursos de forma determinista.

II-C. Generación Aumentada por Recuperación (RAG)

Los sistemas RAG anclan la generación del modelo a fuentes documentales externas, reduciendo alucinaciones factuales al inyectar contexto relevante directamente en el prompt [7]. Estudios posteriores han extendido este paradigma a entornos educativos, donde la recuperación de fragmentos del plan docente condiciona la generación a contenido validado institucionalmente [4]. En el presente proyecto se adopta una variante simplificada: dado que el documento fuente es un único plan docente con estructura predefinida y extensión acotada, el contexto de cada unidad se inyecta directamente en el prompt sin recuperación vectorial, reduciendo la complejidad del sistema sin sacrificar la fidelidad al material original.

II-D. Evaluación automática con LLM-as-a-Judge

El paradigma LLM-as-a-Judge propone utilizar un modelo de lenguaje como evaluador de las salidas generadas por otro modelo, asignando puntajes numéricos por criterio [2]. Validado en el benchmark MT-Bench con alta correlación respecto a evaluaciones humanas expertas, este enfoque automatiza el ciclo de mejora de prompts sin depender exclusivamente de revisores humanos. En el presente proyecto, el evaluador automático recibe cada recurso generado junto con los criterios de calidad definidos —cobertura conceptual, precisión de definiciones, distribución taxonómica de Bloom y viabilidad de la

actividad— y emite un puntaje por criterio con observaciones que orientan el refinamiento de los prompts más débiles.

III. HERRAMIENTAS Y TECNOLOGÍAS

III-A. API de Gemini — *gemini-1.5-flash*

La API de Gemini de Google provee acceso al modelo *gemini-1.5-flash*, seleccionado por su equilibrio entre capacidad de generación de texto estructurado y latencia de respuesta [8]. El acceso se gestiona a través de Google AI Studio con un plan gratuito sin tarjeta de crédito, garantizando la reproducibilidad del proyecto en entornos académicos. La biblioteca *google-generativeai* encapsula las llamadas REST y gestiona la serialización de prompts y respuestas JSON.

III-B. *pypdf*

pypdf es una biblioteca Python de código abierto para extracción de texto de archivos PDF sin dependencias externas. Se utiliza en el módulo *extractor.py* para leer los documentos del plan docente página a página y retornar el texto plano que posteriormente se segmenta por unidad temática en *base_conocimiento.py*.

III-C. *python-docx*

python-docx permite la creación programática de documentos Word (.docx) desde Python. Se emplea en el módulo *exportador.py* para construir el documento final con estilos tipográficos consistentes: portada institucional, encabezados jerárquicos por unidad y tipo de recurso, tablas para el glosario y listas numeradas para las preguntas de comprensión [6].

III-D. *python-dotenv*

python-dotenv gestiona las variables de entorno del proyecto, en particular la clave de la API de Gemini almacenada en un archivo *.env* excluido del repositorio mediante *.gitignore*, garantizando que las credenciales no queden expuestas en el código fuente ni en el historial de versiones de Git.

IV. ARQUITECTURA DEL SISTEMA

La arquitectura del sistema se documenta siguiendo el modelo C4 [9], que describe el sistema en tres niveles de abstracción progresiva: contexto, contenedores y componentes.

IV-A. Nivel 1 — Contexto

El sistema *Generador de Recursos Didácticos* tiene un único actor: el estudiante, que sube los PDFs del plan docente y descarga el documento Word generado. El único sistema externo es la API de Gemini, invocada mediante llamadas REST para ejecutar los cuatro prompts de generación.

IV-B. Nivel 2 — Contenedores

El sistema se compone de cinco contenedores Python con responsabilidades independientes:

1. **Preprocesamiento** (*extractor.py* + *base_conocimiento.py*): extrae el texto de los PDFs y construye el diccionario de unidades.
2. **Pipeline de IA** (*pipeline.py* + *prompts.py*): orquesta las llamadas a Gemini y produce los recursos en JSON.
3. **Almacenamiento JSON intermedio**: archivos *recursos_asig_N_unidad_M.json* que desacoplan la generación de la exportación.
4. **Exportador Word** (*exportador.py*): ensambla el documento Word final desde los JSON.
5. **Evaluación y validación** (*evaluador.py* + *persona*): evalúa la calidad automáticamente y registra el feedback del docente.

IV-C. Nivel 3 — Componentes del Pipeline de IA

Dentro del contenedor Pipeline de IA, los cuatro componentes corresponden a las funciones de *prompts.py*. Cada uno recibe el contexto de la unidad desde la base de conocimiento y escribe su respuesta en el almacenamiento JSON [1]:

1. **generar_resumen()**: síntesis ejecutiva de dos párrafos. Salida: {"parrafo_1": "...", "parrafo_2": "..."}.
2. **generar_glosario()**: 8 a 10 términos técnicos con definición de dos oraciones. Salida: {"terminos": [{"termino": "...", "definicion": "..."}]}.
3. **generar_preguntas()**: cinco preguntas en tres niveles de Bloom con respuesta esperada. Salida: {"preguntas": [{"nivel": "...", "pregunta": "...", "respuesta": "..."}]}.
4. **generar_actividad()**: actividad práctica de 20-30 minutos. Salida: {"enunciado": "...", "producto": "..."}.

IV-D. Nivel 3 — Componentes de Evaluación

Dentro del contenedor de Evaluación, dos componentes trabajan en secuencia: el **Evaluador automático** (*evaluador.py*) aplica LLM-as-a-Judge [2] sobre el documento Word generado y produce un reporte de puntajes por criterio; la **Validación del docente** añade observaciones cualitativas. El feedback combinado retroalimenta el refinamiento de los prompts más débiles en la iteración final.

REFERENCIAS

- [1] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 24824–24837, 2022.
- [2] L. Zheng, W. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Li, Z. Li, E. P. Xing, J. E. Gonzalez, I. Stoica, and H. Zhang, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

- [3] T. H. Kung, M. Cheatham, A. Medenilla, C. Sillos, L. De Leon, C. Elepaño, M. Madriaga, R. Aggabao, G. Diaz-Candido, J. Maningo, and V. Tseng, "Performance of ChatGPT on USMLE: Potential for AI-Assisted Medical Education Using Large Language Models," *PLOS Digital Health*, vol. 2, no. 2, p. e0000198, 2023.
- [4] E. Kasneci, K. Seßler, S. Küchemann, M. Bannert, D. Dementieva, F. Fischer, U. Gasser, G. Groh, S. Günemann, E. Hüllermeier, S. Krusche, G. Kutyniok, T. Michaeli, C. Nerdel, J. Pfeffer, O. Poquet, M. Sailer, A. Schmidt, T. Seidel, M. Stadler, J. Weller, J. Kasneci, and T. Seidel, "ChatGPT for Good? On Opportunities and Challenges of Large Language Models for Education," *Learning and Individual Differences*, vol. 103, p. 102274, 2023.
- [5] S. Min, X. Lyu, A. Holtzman, M. Artetxe, M. Lewis, H. Hajishirzi, and L. Zettlemoyer, "Rethinking the Role of Demonstrations: What Makes In-Context Learning Work?," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 11048–11064, 2022.
- [6] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith, and D. C. Schmidt, "A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT," *IEEE Software*, vol. 41, no. 2, pp. 30–39, 2024.
- [7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [8] G. Team, P. Georgiev, V. I. Lei, R. Burnell, L. Bai, A. Gulati, G. Tanzer, D. Vincent, Z. Pan, S. Wang, et al., "Gemini 1.5: Unlocking Multimodal Understanding Across Millions of Tokens of Context," in *Proc. International Conference on Machine Learning (ICML)*, 2024.
- [9] S. Brown, *The C4 Model for Visualising Software Architecture*. Lean Publishing, 2023.